

Abstract

Civic Pulse is a comprehensive, gamified e-governance web application designed to bridge the communication gap between citizens and municipal authorities. Traditional civic reporting methods are often fragmented, lacking real-time tracking, transparency, and geographic precision. Civic Pulse solves this by providing a centralized platform where citizens can report local infrastructure issues (such as road damage, water leaks, or sanitation problems) using dynamic, interactive GIS mapping (Leaflet.js).

To maximize public participation, the system integrates a gamified reward system where citizens earn points and climb leaderboards for valid reporting. For municipal authorities, the platform offers a secure, role-based Administrative Command Center equipped with real-time data visualization (Chart.js), automated priority sorting, and one-click data exportation. By automating issue tracking and providing clear proof-of-resolution, Civic Pulse aims to foster a smarter, more transparent, and highly responsive smart-city ecosystem.

1. Introduction

1.1 Motivation

The motivation behind this project stems from the inefficiencies and lack of transparency inherent in traditional municipal grievance redressal systems. In most developing urban areas, citizens struggle to report civic issues like broken streetlights, unattended garbage, or severe potholes. They often have to rely on physical paperwork, unmonitored toll-free numbers, or fragmented social media tags. Consequently, citizens feel their voices are unheard, leading to public apathy.

On the administrative side, government authorities are overwhelmed with unorganized data, making it difficult to prioritize critical issues or visualize problem hotspots across the city.

The inspiration for Civic Pulse was to replace this manual, opaque process with an automated, location-aware digital solution. By introducing map-based reporting and a point-based gamification system, the motivation is not only to streamline data management for the government but also to actively inspire and reward citizens for taking ownership of their community's well-being.

1.2 Problem Statement

The existing system for reporting municipal civic issues is manual, fragmented, and lacks geographic precision. Specifically, the current process faces the following problems:

- **Time-Consuming & Error-Prone:** Physical paperwork and phone complaints often lead to lost data and delayed actions.
- **Lack of Transparency:** Citizens have no way to track the real-time status of their reported issues.
- **Poor Data Management:** Administrators lack a centralized dashboard to visualize problem hotspots and prioritize critical issues. Hence, there is a critical need for an automated, location-aware, and transparent grievance redressal system.

1.3 Purpose / Objective and Goals

Objectives:

- To automate the existing manual civic reporting system using modern web technologies (Flask, Python, JavaScript).
- To integrate interactive GIS mapping (Leaflet.js) to pinpoint the exact geographic coordinates of reported issues.
- To implement a secure, role-based architecture separating Citizen functionalities from Administrative command controls.
- To introduce a gamified reward system (points and ranks) to actively incentivize public participation.

Goals:

- Develop a highly user-friendly, responsive web application accessible to citizens with varying levels of technical expertise.
- Provide accurate, quick, and transparent status tracking for every reported grievance (Pending, In Progress, Resolved).
- Improve overall municipal efficiency by allowing administrators to filter complaints by status, priority, and year.
- Establish a reliable, secure, and scalable e-governance platform that bridges the gap between the public and the government.

1.4 Literature Survey

Various existing civic reporting systems and e-governance platforms were analyzed to understand current methodologies:

- **Traditional Methods:** Traditional toll-free numbers or physical grievance forms are highly time-consuming, prone to data loss, and offer no real-time status tracking for the citizen.
- **Generic Web Forms & Social Media:** Citizens often use social media (e.g., X/Twitter) or basic government web forms to complain. While accessible, these methods lack structured geographic data (GIS mapping) and do not automatically organize issues by priority for the authorities.
- **Lack of Incentive:** Most existing municipal portals are one-way communication channels. They do not reward or incentivize citizens, resulting in low public engagement.
- **The Gap:** Very few applications successfully combine precise interactive GIS location tracking, real-time transparency, and a gamified reward system into a single, dedicated platform.

1.5 Project Scope and Limitations

Scope:

- Designed for local citizens and municipal administrators.
- Two-tier role system: User (Citizen) and Admin (Authority).
- Interactive, map-based complaint registration capturing exact latitude/longitude coordinates (using Leaflet.js).
- Gamified reward system assigning points and dynamic ranks to citizens for active participation.
- Secure Administrative dashboard for filtering, visualizing (Chart.js), and resolving reports.
- One-click CSV data exportation for municipal record-keeping.

Limitations:

- Strictly requires an active internet connection to load live maps and synchronize with the database.
- The accuracy of the "Locate Me" feature depends entirely on the user's device hardware (mobile GPS is highly accurate, whereas desktop Wi-Fi location can be approximate).
- The current version does not feature AI-powered image verification to check if an uploaded photo is a genuine civic issue or spam.

2. System Analysis

2.1 Existing System

Currently, municipal grievance management relies heavily on legacy frameworks. When infrastructure or sanitation issues occur, the reporting process is localized through physical application forms, unmonitored toll-free numbers, or disjointed email threads. These traditional methods operate in information silos, meaning complaint data is not centralized. Furthermore, there is a severe lack of spatial intelligence (geographic tracking), meaning municipal workers often struggle to locate the exact site of the problem, and citizens receive zero real-time tracking updates regarding their submissions.

2.2 Scope and Limitations of Existing System

Scope

- Accepts citizen complaints primarily through toll-free phone calls, physical paper applications at the municipal office, or basic email.
- Stores public grievance data manually in physical paper registers or unshared, offline Excel spreadsheets.
- Relies on manual, verbal instructions to dispatch municipal workers to a general text-based address.

Limitations

- **No Geographic Precision (No Maps):** Because citizens can only provide text-based addresses (e.g., "near the main square"), field workers waste significant time trying to find the exact pothole or broken pipe.
- **Lost and Ignored Data:** Physical paperwork and disorganized email inboxes lead to a high rate of misplaced, delayed, or completely ignored citizen complaints.
- **No Citizen Feedback Loop:** Once a citizen registers a complaint, they have no way to track its real-time status. They do not know if it is "Pending" or "In Progress," leading to frustration and repeated calls.
- **No Proof of Work:** Higher-up authorities cannot easily verify if a problem was actually fixed by the ground workers, as there is no system for capturing and uploading post-work image proof.

2.3 Project Perspective and Features

Project Perspective:

Civic Pulse is envisioned as a next-generation e-governance platform that replaces legacy workflows with an automated, spatially-aware digital command center. It acts as a transparent bridge between the community and the government, driven by real-time tracking and gamified public participation.

Key Features:

- **Secure Role-Based Access Control (RBAC):** Strict isolation between Citizen interfaces and Administrative command controls.
- **Interactive GIS Mapping:** Real-time location pinning using Leaflet.js to capture exact geographic coordinates.
- **Dynamic Gamification Engine:** A point-based reward system that automatically updates citizen ranks (Activist, Guardian, Hero) based on valid reporting.
- **Real-Time Visual Analytics:** An administrative dashboard powered by Chart.js for tracking resolution rates and category distributions.
- **Visual Proof of Resolution:** Administrators can upload post-work images as official proof, viewable by citizens via an interactive lightbox.

2.4 Stakeholders

The primary individuals and groups who interact with and benefit from the system include:

- **Citizens (End-Users):** Local residents utilizing the platform to report and track neighbourhood infrastructure issues.
- **Municipal Administrators:** Command center operators who evaluate, prioritize, and update the status of incoming reports.
- **On-Ground Maintenance Crews:** Field technicians who are dispatched using the highly accurate geospatial map data to resolve issues.
- **Urban Planners:** City officials who can observe the dashboard analytics to identify recurring problem zones across the city.

2.5 Requirement Analysis

Functional Requirements:

- User Registration and Login Authentication.
- Submit map-based complaints with geographic coordinates and image evidence.
- Track real-time complaint status and download official PDF receipts.
- Admin command center to filter, review, and update report statuses.
- Automated allocation or deduction of citizen reward points based on report validity.

Performance Requirements:

- Render interactive GIS maps and analytical charts within 2-3 seconds.
- Support concurrent report submissions from multiple citizens smoothly.
- Maintain a highly responsive UI/UX across both desktop and mobile devices.

Security Requirements:

- Cryptographic password hashing (using Werkzeug.security) for all user accounts.
- Strict session validation to prevent unauthorized access to Administrative endpoints.
- Backend file validation to restrict malicious file uploads (allowing only specific image formats).

3. System Design

3.1 Design Constraints

During the design and development of **Civic Pulse**, several constraints were considered to ensure the system remained feasible, secure, and accessible:

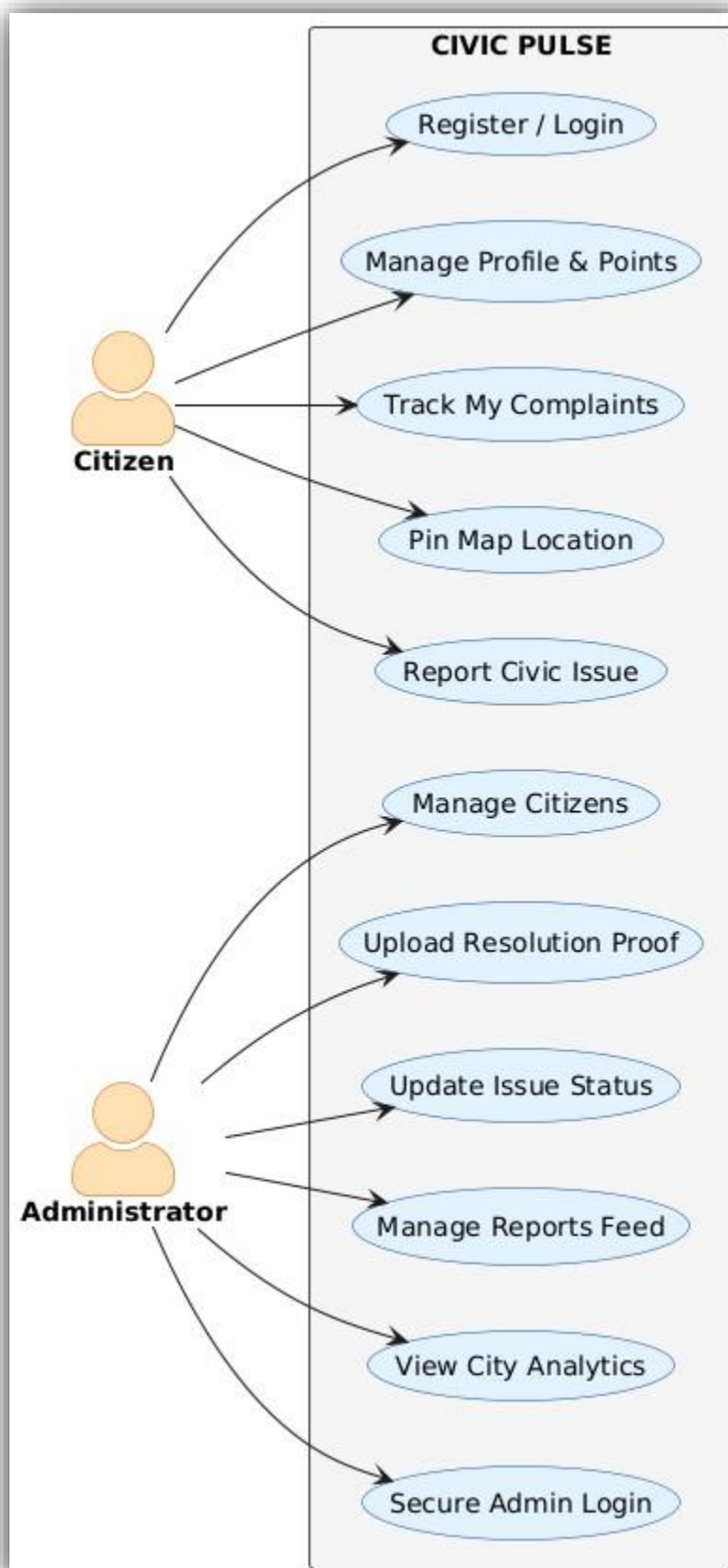
- **Technological Constraints:** The system relies on a specific web technology stack. The frontend strictly requires HTML5, CSS3, and JavaScript, while the backend is constrained to Python (Flask) and PostgreSQL. Users must have a modern, JavaScript-enabled web browser to use the interactive maps.
- **Hardware/Location Constraints:** The accuracy of the interactive map pinning is constrained by the user's device hardware. Mobile devices with GPS provide high accuracy, whereas desktop Wi-Fi location can be approximate.
- **Time Constraints:** The project had to be designed, developed, and tested within a fixed academic timeframe, prioritizing core functional requirements over secondary features.
- **Financial Constraints:** To keep the project cost-effective, free-tier APIs (like OpenStreetMap for Leaflet.js) and open-source libraries were utilized instead of paid enterprise software.

3.2 System Model (OOSE)

Object-Oriented Software Engineering (OOSE) principles are applied to structure the system's behavior and logic. The system interactions and structures are visualized through the following core diagrams:

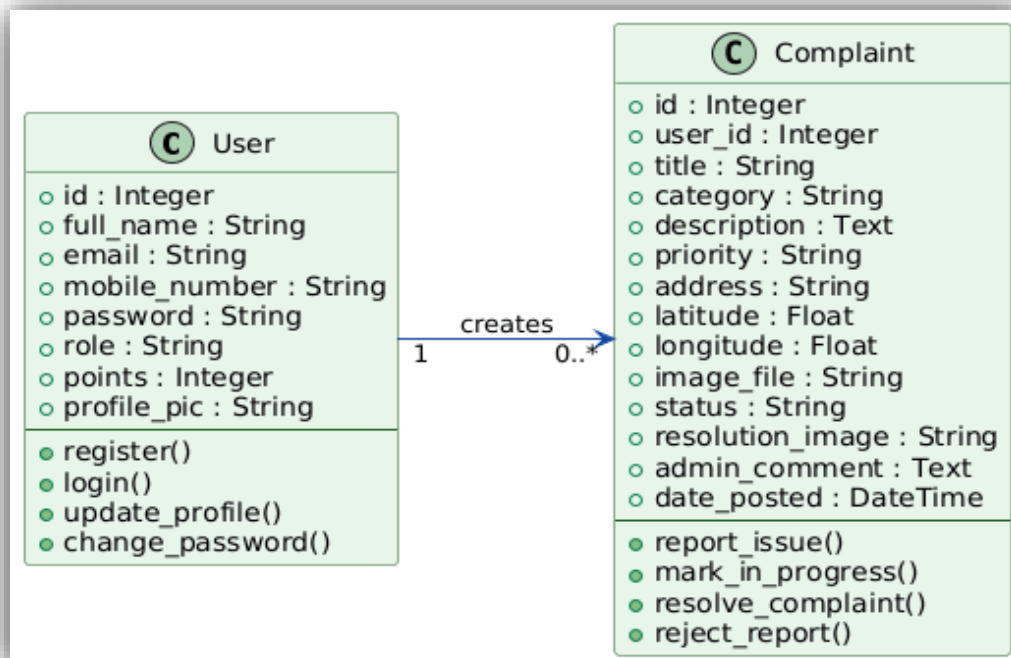
1. Use Case Diagram

The Use Case diagram illustrates the behavioral requirements of the Civic Pulse platform from the perspective of its two primary actors: the Citizen and the Administrator.



2. Class Diagram

The Class diagram outlines the static structure of the application, detailing the core software objects (Users and Complaints), their specific attributes, and essential operations based on the Python backend implementation.

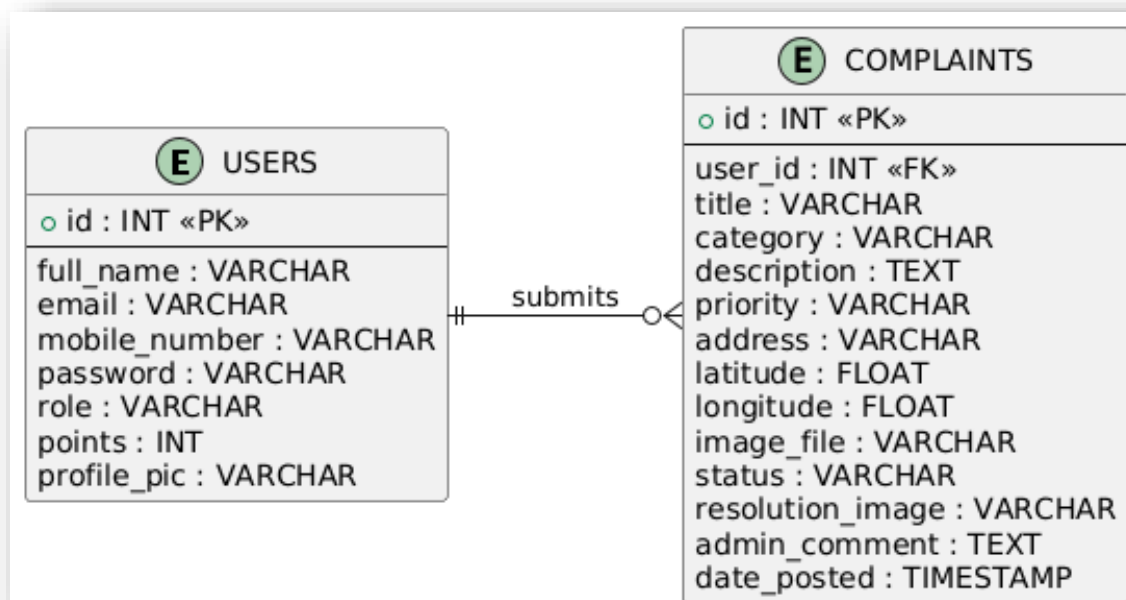


3.3 Data Model

The backend architecture is built upon a robust relational database (PostgreSQL via SQLAlchemy). The Data Model details exactly how information is persistently stored and related.

1. Entity Relationship (ER) Diagram

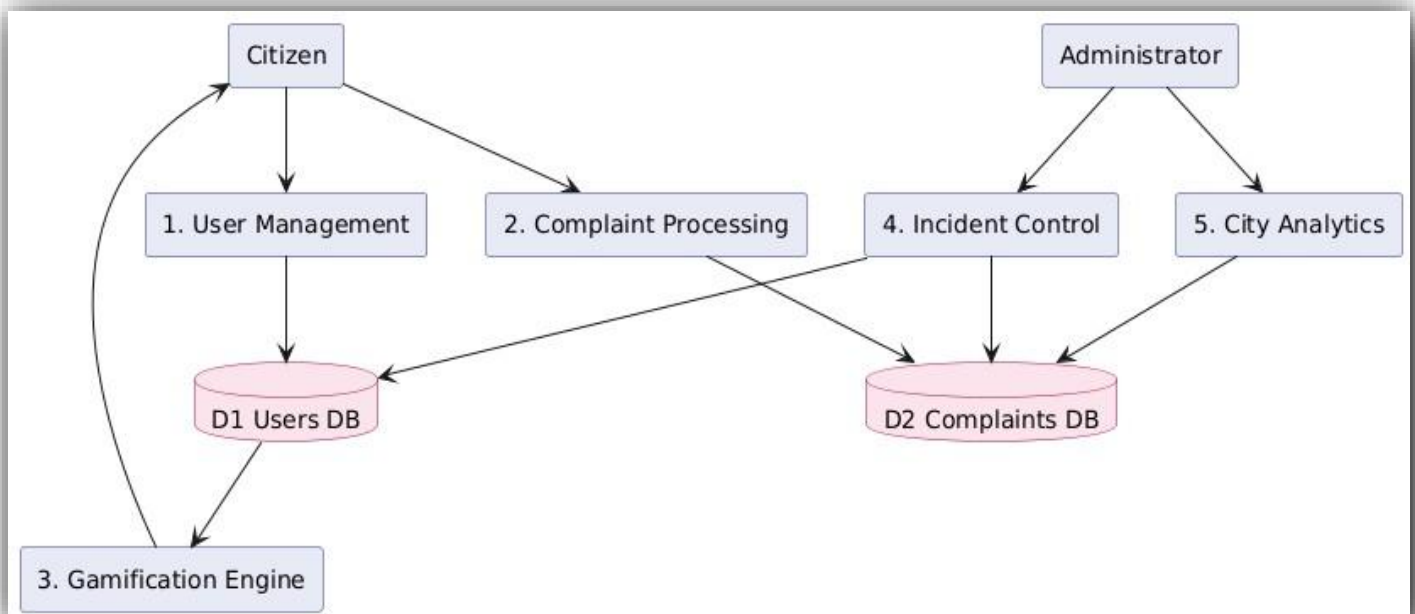
The ER diagram maps the database tables and their structural links, emphasizing the fundamental One-to-Many relationship between users and their reported complaints.



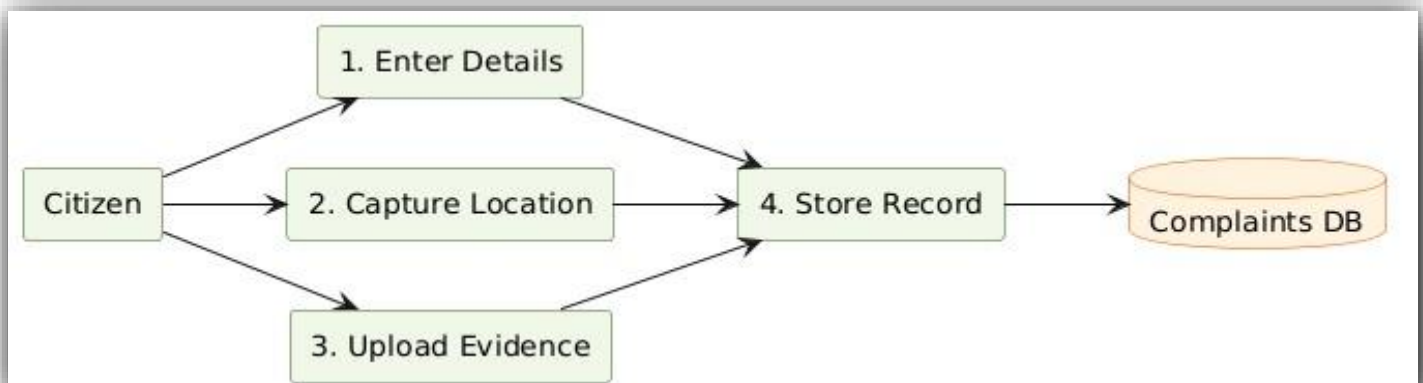
DATA FLOW DIAGRAM LEVEL – 0:



DATA FLOW DIAGRAM LEVEL – 1:



DATA FLOW DIAGRAM LEVEL – 2:



2. Tables, Attributes, and Relationship

Table: USERS

Column Name	Data Type	Key / Constraint	Description
id	INT AUTO_INCREMENT	Primary Key	Unique identifier for the user
full_name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(100)	UNIQUE, NOT NULL	Login email address
mobile_number	VARCHAR(15)	NULL	Contact number
password	VARCHAR(255)	NOT NULL	Hashed password
role	VARCHAR(20)	DEFAULT 'user'	Citizen or Admin role
points	INT	DEFAULT 0	Gamification points
profile_pic	VARCHAR(255)	NULL	Avatar image path

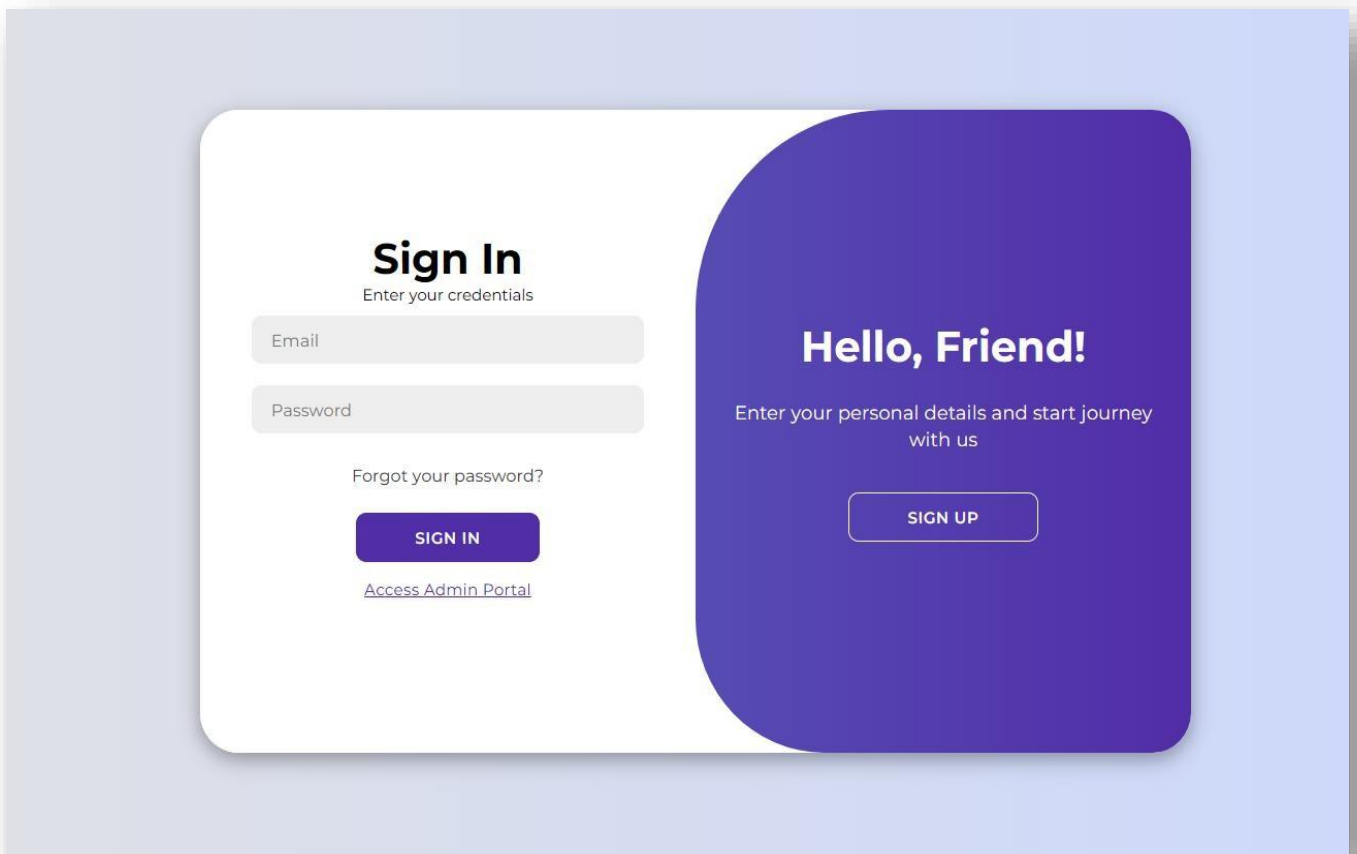
Table: COMPLAINTS

Column Name	Data Type	Key / Constraint	Description
id	INT AUTO_INCREMENT	Primary Key	Unique complaint ID
user_id	INT	Foreign Key → users(id)	Links complaint to user
title	VARCHAR(150)	NOT NULL	Issue title
category	VARCHAR(100)	NOT NULL	Complaint category
description	TEXT	NOT NULL	Detailed description
priority	VARCHAR(20)	NOT NULL	Low / Medium / High
address	VARCHAR(255)	NOT NULL	Issue location address
latitude	FLOAT	NOT NULL	Geographic latitude
longitude	FLOAT	NOT NULL	Geographic longitude
image_file	VARCHAR(255)	NULL	Uploaded evidence
status	VARCHAR(20)	DEFAULT 'Pending'	Current complaint state
resolution_image	VARCHAR(255)	NULL	Admin resolution proof
admin_comment	TEXT	NULL	Admin remarks
date_posted	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Auto-generated timestamp

3.4 User Interfaces

To provide a clear understanding of the system's frontend design, the following screenshots demonstrate the core Graphical User Interfaces (GUI) of Civic Pulse:

1. Authentication Interface



```
@app.route('/login-api', methods=['POST'])
def login_api():
    data = request.get_json()
    user = User.query.filter_by(email=data['email']).first()
    login_mode = data.get('mode')

    if user and check_password_hash(user.password, data['password']):
        if user.role == 'admin' and login_mode and login_mode != 'admin':
            return jsonify({'message': 'Access Denied. Please use the Admin Portal.'}), 403
        if user.role != 'admin' and login_mode == 'admin':
            return jsonify({'message': 'Access Denied. You are not an Admin.'}), 403

        session['user_id'] = user.id
        session['role'] = user.role

        if user.role == 'admin':
```

Welcome Back!

To keep connected with us please login with your personal info

[SIGN IN](#)

Create Account

Use your email for registration

Full Name

Email

Mobile Number

Password

[SIGN UP](#)

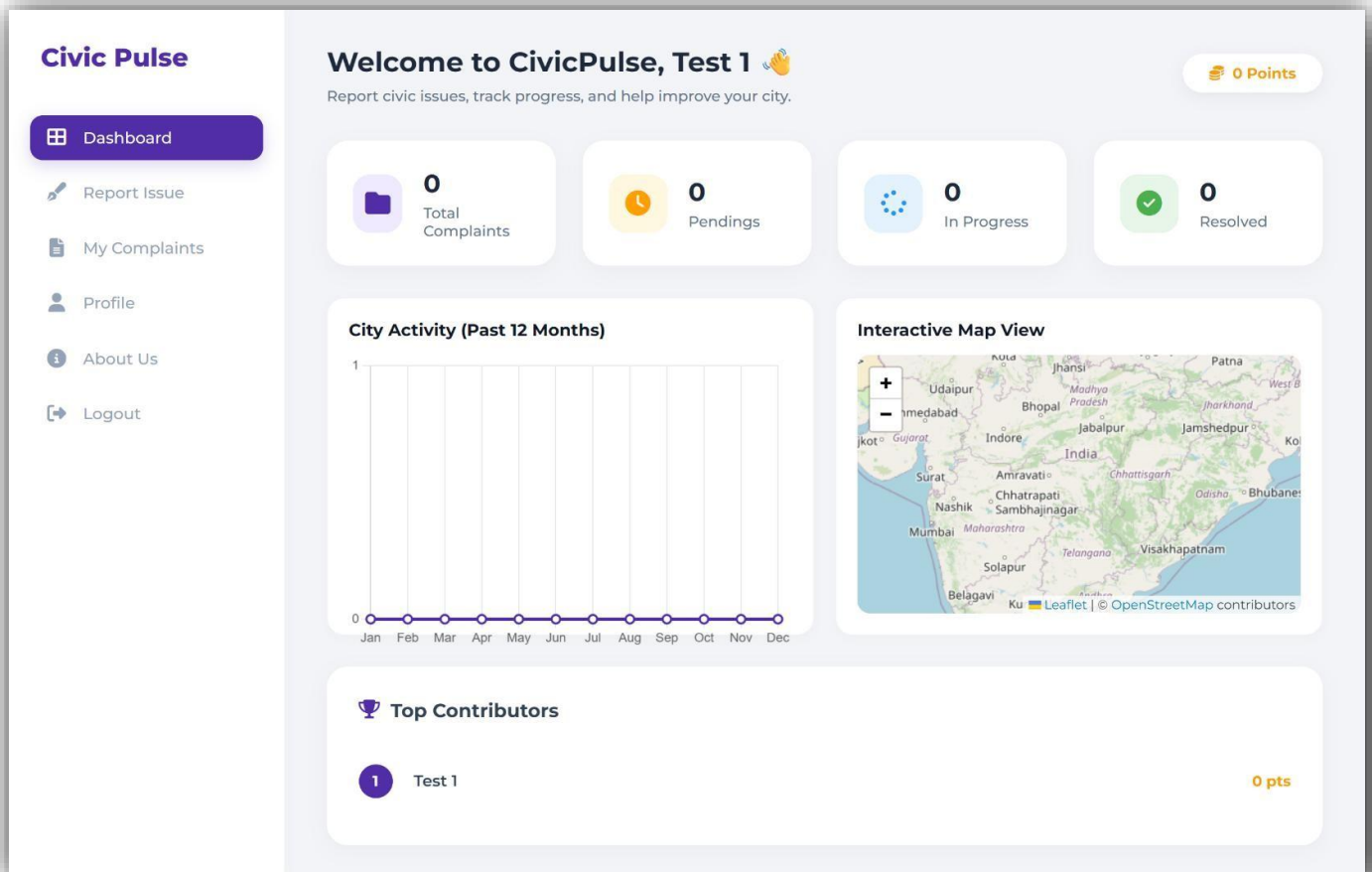
```
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()

    if User.query.filter_by(email=data['email']).first():
        return jsonify({'message': 'Email exists'}), 400

    role = 'user'
    if data['email'] == 'admin@civicpulse.com':
        role = 'admin'

    new_user = User(
        full_name=data['full_name'],
        email=data['email'],
        mobile_number=data['mobile_number'],
        password=generate_password_hash(data['password']),
        role=role
    )
    db.session.add(new_user)
    db.session.commit()
    return jsonify({'message': 'Success'}), 201
```

2. Citizen Dashboard



```
@app.route('/dashboard')
def dashboard():
    if 'user_id' not in session: return redirect(url_for('index'))
    user = User.query.get(session['user_id'])

    stats = {
        'total': Complaint.query.count(),
        'pending': Complaint.query.filter_by(status='Pending').count(),
        'progress': Complaint.query.filter_by(status='In Progress').count(),
        'resolved': Complaint.query.filter_by(status='Resolved').count()
    }

    leaderboard = User.query.filter(User.role != 'admin').order_by(User.points.desc()).limit(3).all()

    return render_template('dashboard.html', user=user, stats=stats, leaderboard=leaderboard)
```

3. Issue Reporting Module

Civic Pulse

Dashboard

Report Issue

My Complaints

Profile

About Us

Logout

Submit a Report 📌

Select a category and pin the location on the map.

0 Points

Select Issue Category

Pothole

Garbage

Water Leak

Street Light

Drainage

Other

Issue Details

Issue Title

e.g. Broken Pipe at Main St

Priority

High

Medium

Low

Description

Provide more details or Address ...

Upload Evidence

Drag & Drop or Click to Upload

Pin Location

Locate Me

Selected Address...

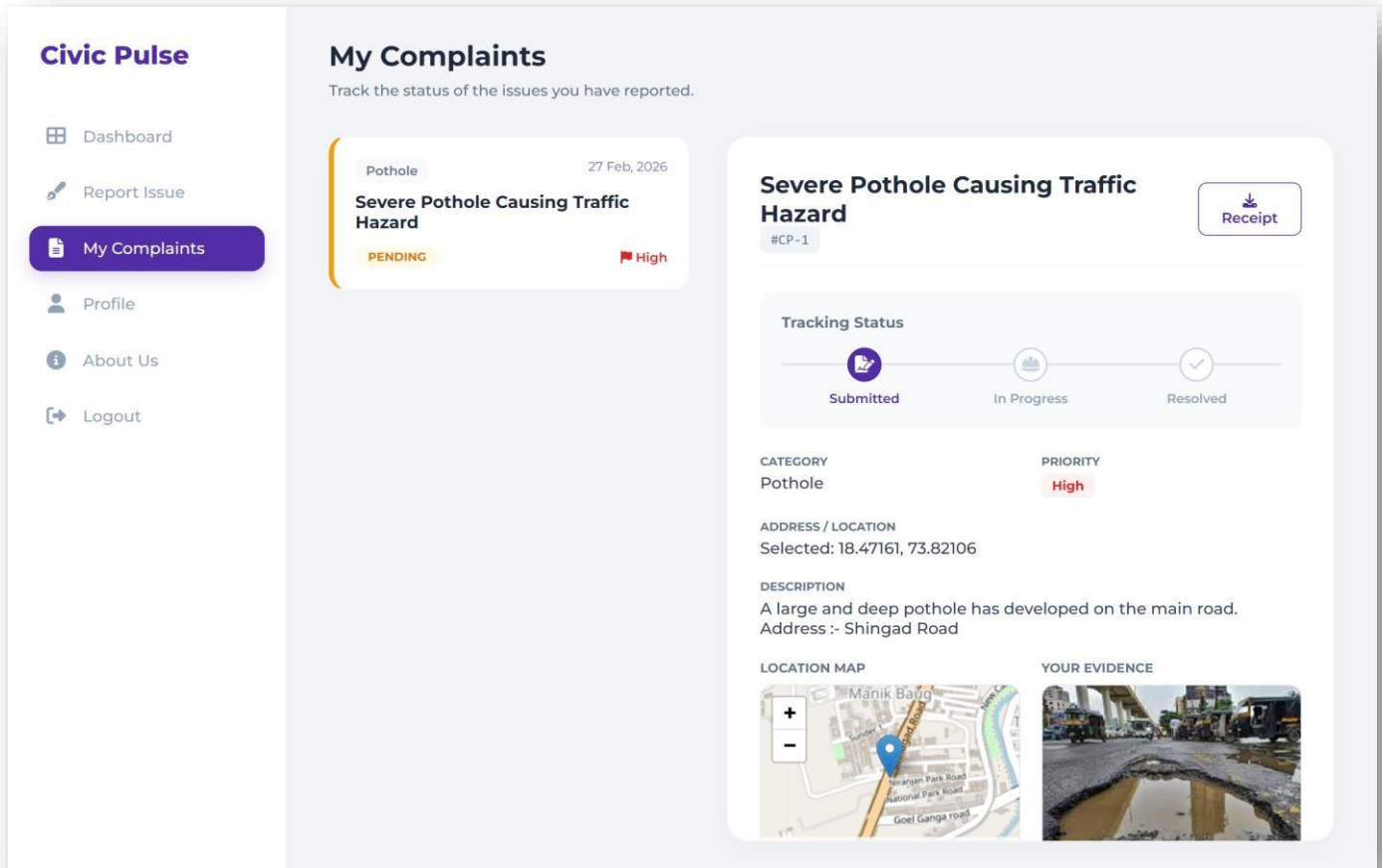
Submit Report to Authorities

```
@app.route('/report-issue', methods=['GET', 'POST'])
def report_issue():
    user = User.query.get(session['user_id'])

    if request.method == 'POST':
        file = request.files.get('image')
        filename = 'default.jpg'
        if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            file.save(os.path.join(app.root_path, app.config['UPLOAD_FOLDER'], filename))

        new_complaint = Complaint(
            category=request.form.get('category'),
            title=request.form.get('title'),
            priority=request.form.get('priority'),
            address=request.form.get('address'),
            latitude=request.form.get('latitude') or None,
            longitude=request.form.get('longitude') or None,
            image_file=filename,
            user_id=session['user_id']
        )
        user.points += 10 # Reward gamification points
        db.session.add(new_complaint)
        db.session.commit()
        return redirect(url_for('dashboard'))
    return render_template('report_issue.html', user=user)
```

4. Citizen Complaint Tracking Interface



```
@app.route('/my-complaints')
def my_complaints():
    if 'user_id' not in session: return redirect(url_for('index'))
    user = User.query.get(session['user_id'])

    # Fetch only the complaints submitted by the logged-in user
    my_reports = Complaint.query.filter_by(user_id=user.id).order_by(Complaint.date_posted.desc()).all()

    return render_template('my_complaints.html', user=user, complaints=my_reports)
```

5. Citizen Profile

Civic Pulse

- Dashboard
- Report Issue
- My Complaints
- Profile**
- About Us
- Logout

My Profile

Manage your account settings and view your impact.



Test 1
 Citizen

Level Progress 10 / 100
90 XP to next level

Personal Details

FULL NAME	EMAIL
Test 1	test1@com
MOBILE NUMBER	LOCATION
1234	Pune, India

Save ChangesChange Password

```
@app.route('/profile', methods=['GET', 'POST'])
def profile():
    user = User.query.get(session['user_id'])

    # Calculate Gamification Rank based on earned points
    rank, target = "Citizen", 100
    if user.points < 100: rank, target = "Citizen", 100
    elif user.points < 500: rank, target = "Activist", 500
    elif user.points < 1000: rank, target = "Guardian", 1000
    else: rank, target = "Hero", 5000

    progress = (user.points / target) * 100
    return render_template('profile.html', user=user, rank=rank, progress=progress, target=target)
```

Civic Pulse

- Dashboard
- Report Issue
- My Complaints
- Profile
- About Us
- Logout

About Us 🚀

Building smarter cities, one report at a time.

🏆 10 Points

We Are Civic Pulse

A next-generation platform empowering citizens to take charge of their city's infrastructure. We bridge the gap between problems and solutions using technology.



🚩 Report Issues

🏆 Earn Points

🏡 Upgrade Your City

Why Choose Us?



Lightning Fast

Report issues in under 30 seconds. Our geo-tagging tech ensures pin-point accuracy.



Transparency

Track every step. From "Submitted" to "Resolved", you see exactly what the admin sees.

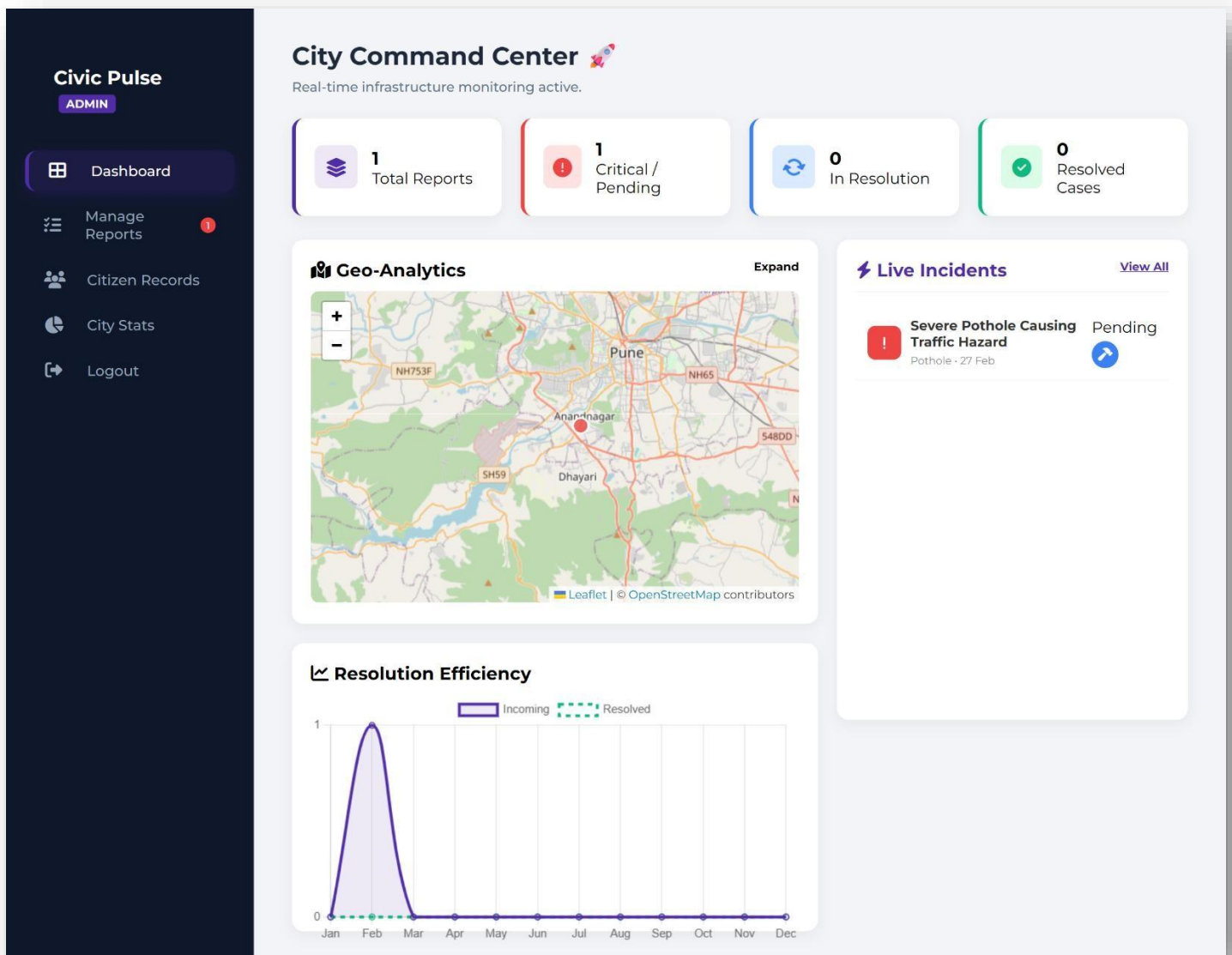


Gamified Impact

Earn XP for every report. Climb the ranks from Citizen to Hero and win rewards.

© 2026 Civic Pulse Initiative. All Rights Reserved.
Designed with ❤️ for a Better Tomorrow.

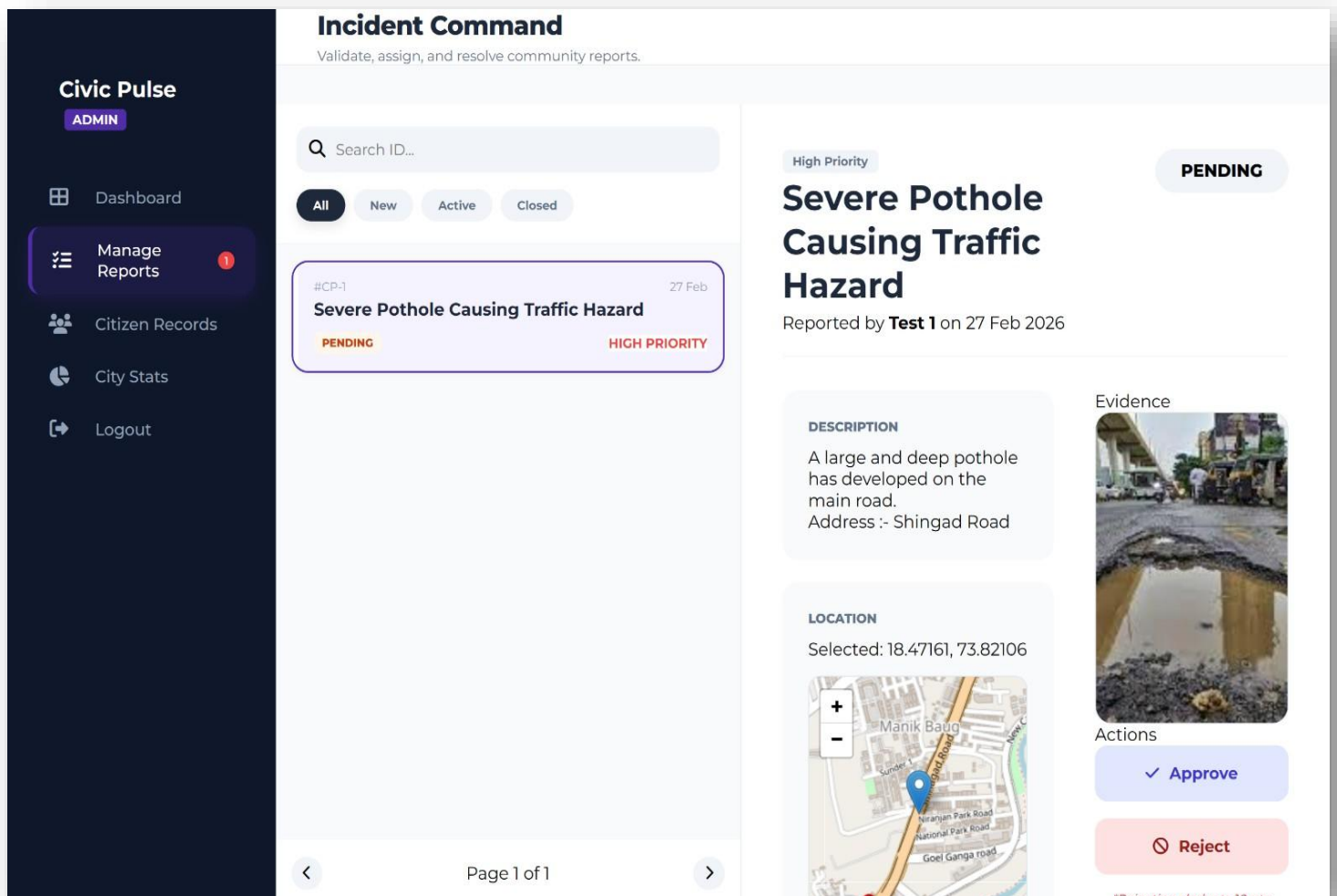
6. Admin Incident Command and Analytics



```
@app.route('/admin/dashboard')
def admin_dashboard():
    # Fetch all complaints city-wide for the admin feed
    all_complaints = Complaint.query.order_by(Complaint.date_posted.desc()).all()

    stats = {
        'total': Complaint.query.count(),
        'pending': Complaint.query.filter_by(status='Pending').count(),
        'progress': Complaint.query.filter_by(status='In Progress').count(),
        'resolved': Complaint.query.filter_by(status='Resolved').count()
    }
    return render_template('admin/dashboard.html', complaints=all_complaints, stats=stats)
```

7. Report Management Console

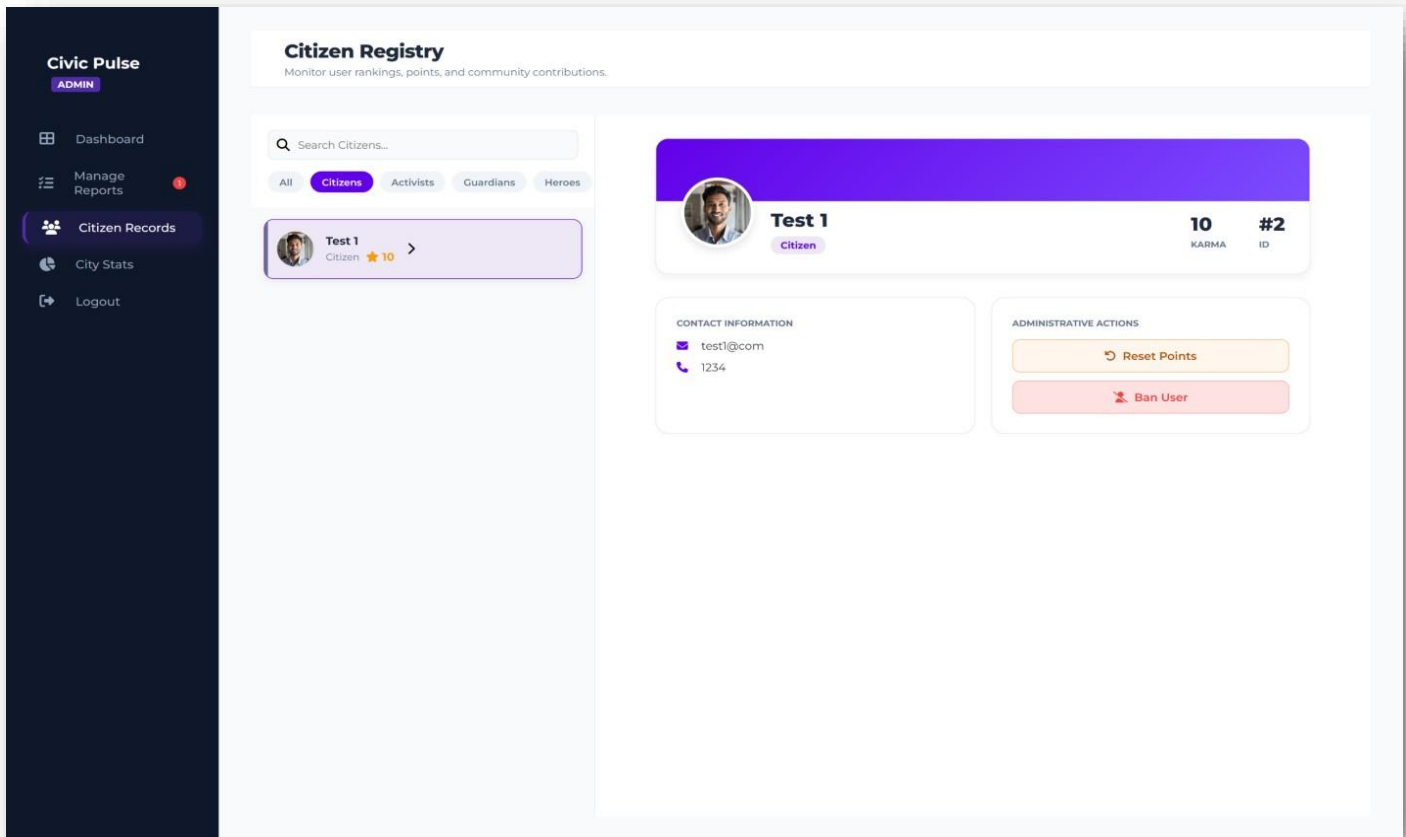


```
@app.route('/admin/manage-reports')
def admin_manage_reports():
    search_query = request.args.get('search')
    query = Complaint.query

    if search_query:
        search = f"%{search_query}%"
        query = query.join(User).filter(
            (Complaint.title.ilike(search)) |
            (Complaint.id.astype(str).ilike(search))
        )

    complaints = query.order_by(Complaint.date_posted.desc()).all()
    return render_template('admin/manage_reports.html', complaints=complaints)
```

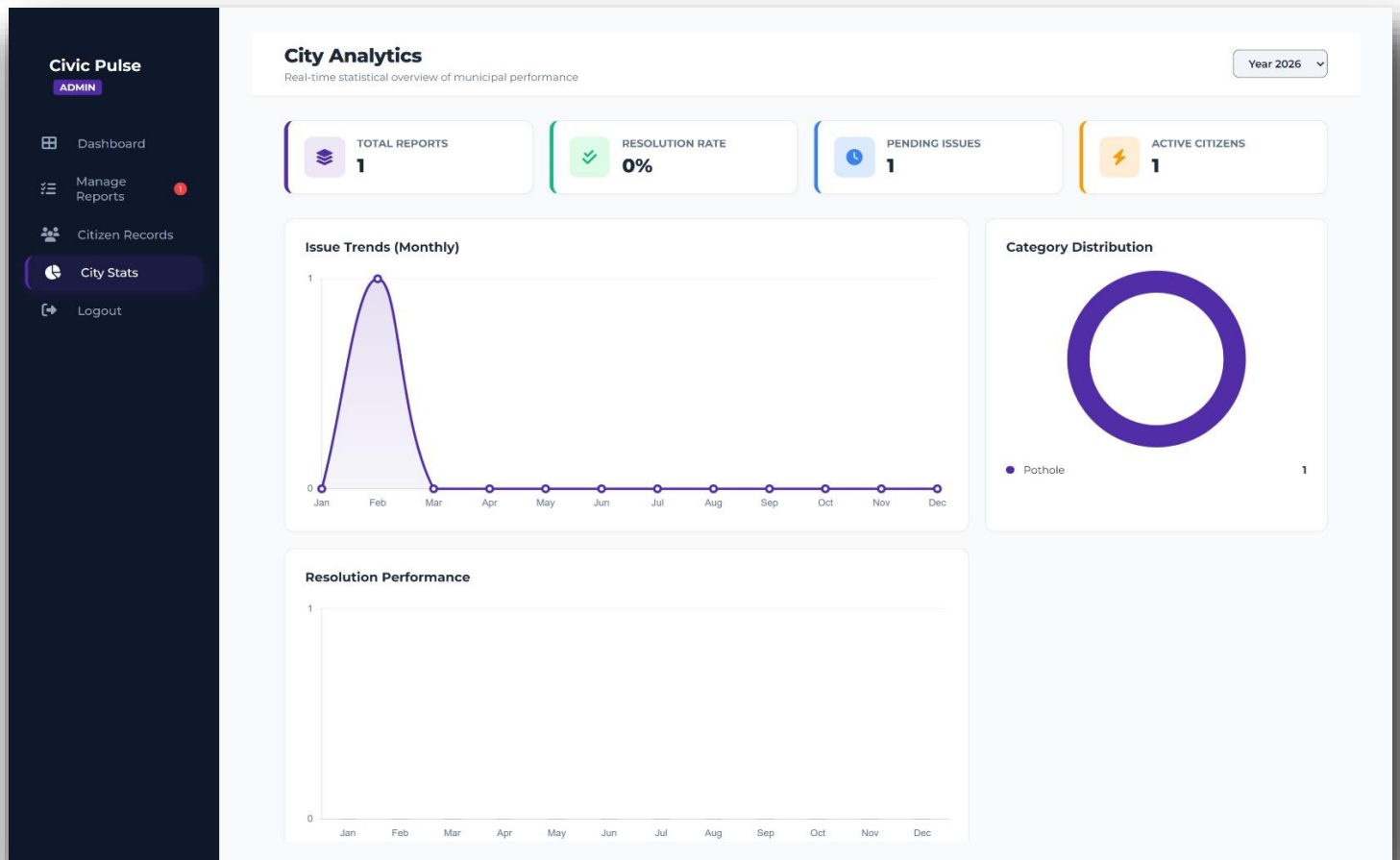
8. Citizen Records



```
@app.route('/admin/citizens')
def admin_citizens():
    # Fetch all citizens ordered by their gamification points
    citizens = User.query.filter(User.role != 'admin').order_by(User.points.desc()).all()
    return render_template('admin/citizen_records.html', citizens=citizens)

@app.route('/admin/user/ban/<int:user_id>', methods=['POST'])
def ban_user(user_id):
    user = User.query.get_or_404(user_id)
    Complaint.query.filter_by(user_id=user.id).delete()
    db.session.delete(user)
    db.session.commit()
    return jsonify({'message': 'User permanently removed.'})
```

9. City Analytics



```
@app.route('/admin/stats')
def admin_stats():
    selected_year = request.args.get('year', datetime.utcnow().year, type=int)

    categories = db.session.query(Complaint.category, func.count(Complaint.id))\
        .filter(func.extract('year', Complaint.date_posted) == selected_year)\
        .group_by(Complaint.category).all()

    active_citizens = User.query.filter(User.points > 0).count()

    return render_template('admin/city_stats.html', categories=categories,
        active_citizens=active_citizens)
```

4. Implementation Details

4.1 Software Specifications

The development and deployment of the Civic Pulse platform utilized a robust, open-source technology stack to ensure scalability, security, and responsive performance.

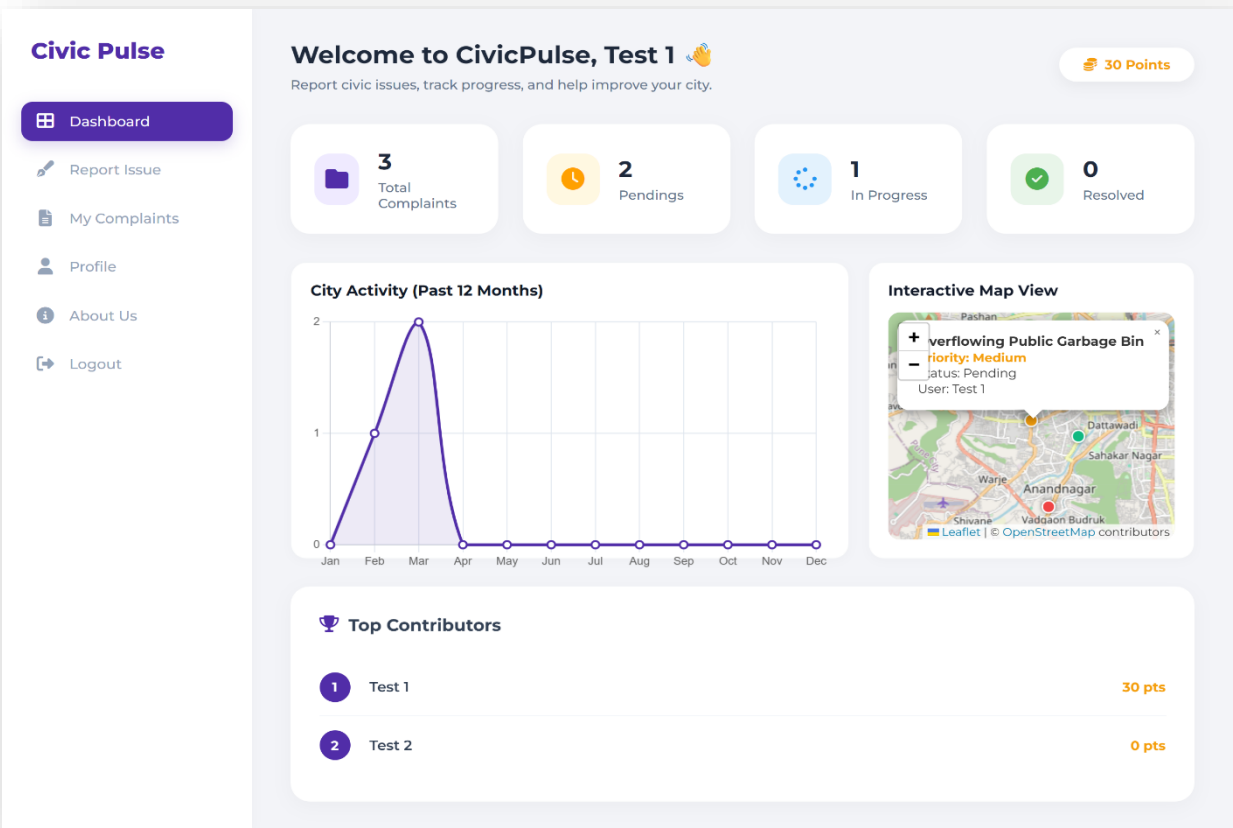
- Operating System: Windows 10/11, macOS, or Linux (Ubuntu)
- Frontend Technologies: HTML5, CSS3, and modern JavaScript (ES6+)
- Frontend Libraries: * Leaflet.js: For interactive GIS mapping and geospatial coordinate capture.
 - Chart.js: For rendering dynamic administrative visual analytics.
- Backend Framework: Python 3.x with Flask
- Database Management System: PostgreSQL
- Database ORM: SQLAlchemy (for structured database queries and management)
- Security & Authentication: Werkzeug.security (for cryptographic password hashing and salting)
- Development Environment (IDE): Visual Studio Code (VS Code)
- Web Browser Requirement: Any modern, JavaScript-enabled browser (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge)

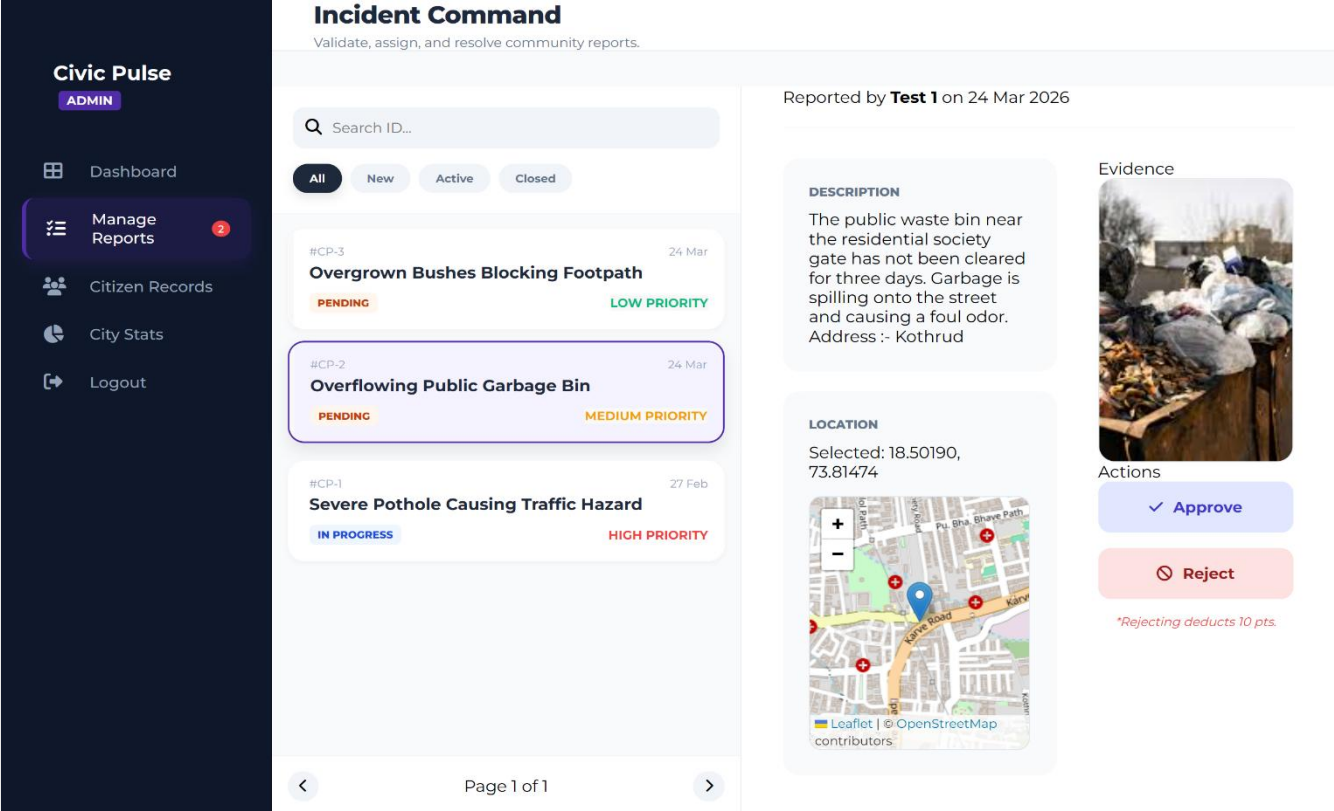
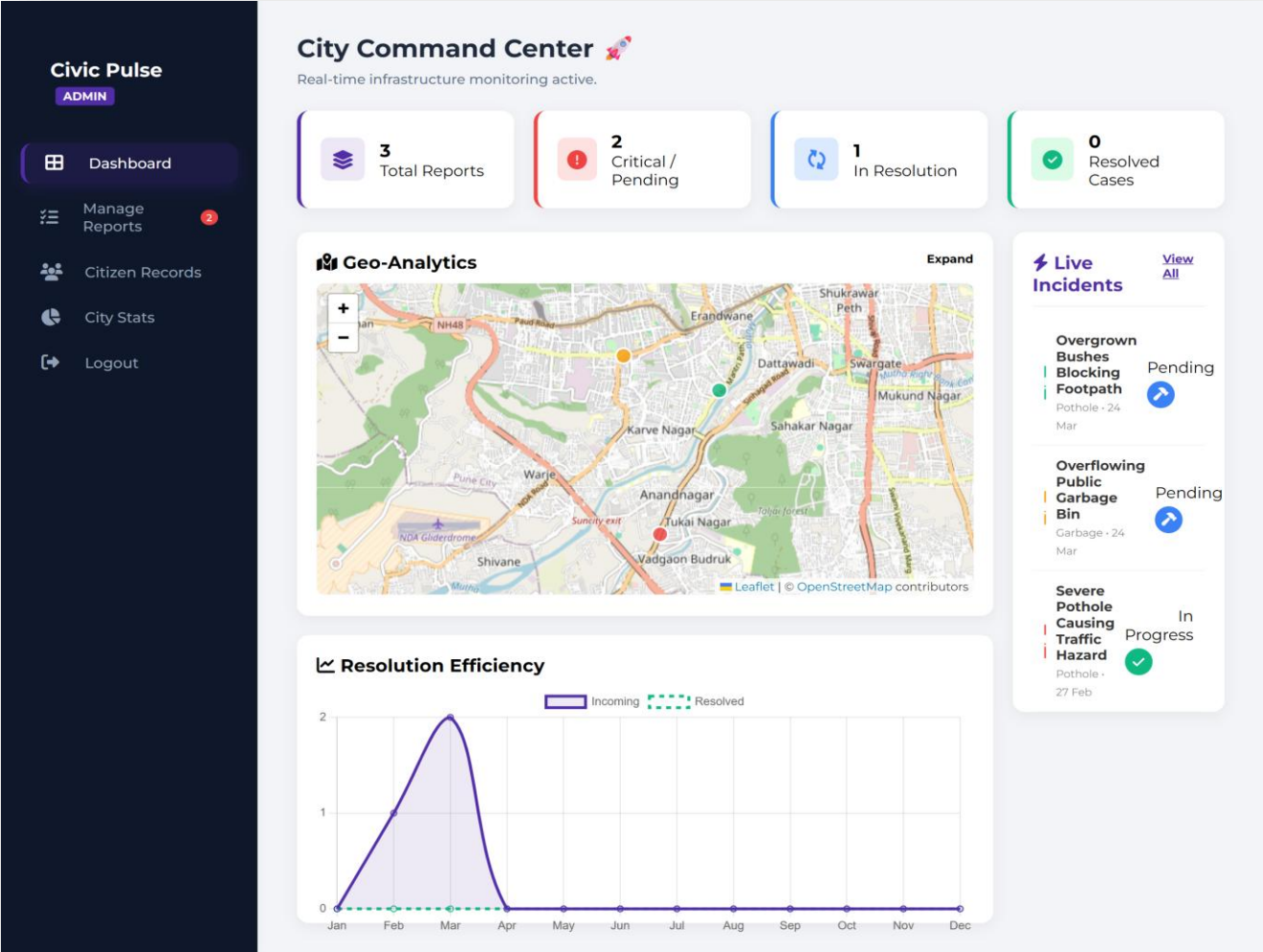
4.2 Hardware Specifications

To successfully host, develop, and run the Civic Pulse web application, the following minimum hardware specifications are required:

- Processor: Intel Core i3 (or AMD equivalent) or higher
- RAM (Memory): Minimum 4 GB (8 GB recommended for smooth concurrent processing)
- Storage (Hard Disk/SSD): Minimum 250 GB total storage (with at least 500 MB of free space dedicated to the project files, database records, and uploaded citizen evidence images)
- Network: An active, stable broadband Internet connection (required for fetching live OpenStreetMap tiles and asynchronous data loading)
- Location Hardware: A device with an active GPS module or Wi-Fi location services enabled (for the "Locate Me" frontend feature to accurately pin civic issues)

5. Outputs and Reports





Civic Pulse

ADMIN

Dashboard

Manage Reports1

Citizen Records

City Stats

Logout

Incident Command

Validate, assign, and resolve community reports.

Search ID...

All

New

Active

Closed

#CP-224 Mar

Overflowing Public Garbage Bin

IN PROGRESSMEDIUM PRIORITY

#CP-127 Feb

Severe Pothole Causing Traffic Hazard

IN PROGRESSHIGH PRIORITY

DESCRIPTION

The public waste bin near the residential society gate has not been cleared for three days. Garbage is spilling onto the street and causing a foul odor. Address :- Kothrud

LOCATION

Selected: 18.50190, 73.81474

Evidence

Actions

Resolve

<

Page 1 of 1

>

Confirm Resolution

Upload proof to notify citizen.

Remarks

The solid waste management team has been notified and a cleanup crew has been dispatched to this location.

Proof

Upload Image

Mark Resolved (+50 pts)

Civic Pulse

ADMIN

Dashboard

Manage Reports

Citizen Records

City Stats

Logout

Incident Command

Validate, assign, and resolve community reports.

Q Search ID...

All

New

Active

Closed

#CP-3

24 Mar

Overgrown Bushes Blocking Footpath

REJECTED

#CP-2

24 Mar

Overflowing Public Garbage Bin

IN PROGRESS

MEDIUM PRIORITY

#CP-1

27 Feb

Severe Pothole Causing Traffic Hazard

IN PROGRESS

HIGH PRIORITY

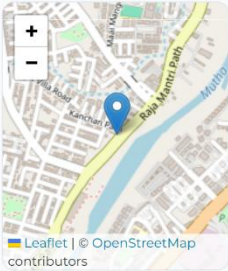
Reported by Test 1 on 24 Mar 2026

DESCRIPTION

Minor vegetation from the local park is growing through the fence and slightly blocking the walking path for pedestrians.

LOCATION

Selected: 18.49606, 73.83141



Evidence



Actions

Rejected

<

Page 1 of 1

>

Civic Pulse

ADMIN

Dashboard

Manage Reports

Citizen Records

City Stats

Logout

City Analytics

Real-time statistical overview of municipal performance

Year 2026

TOTAL REPORTS

3

RESOLUTION RATE

33%

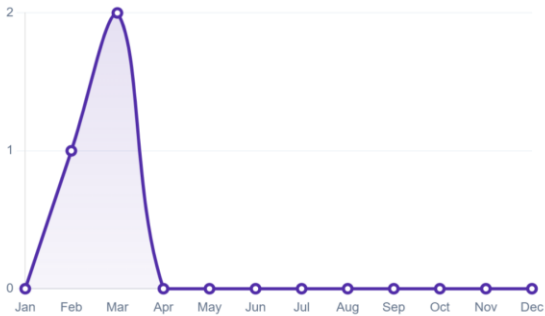
PENDING ISSUES

0

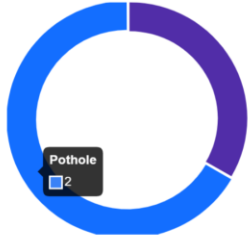
ACTIVE CITIZENS

1

Issue Trends (Monthly)



Category Distribution



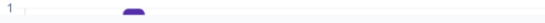
Garbage

1

Pothole

2

Resolution Performance



Civic Pulse

Dashboard

Report Issue

My Complaints

Profile

About Us

Logout

My Complaints

Track the status of the issues you have reported.

Pothole 24 Mar, 2026

Overgrown Bushes Blocking Footpath

REJECTED

Low

Garbage 24 Mar, 2026

Overflowing Public Garbage Bin

RESOLVED

Medium

Pothole 27 Feb, 2026

Severe Pothole Causing Traffic Hazard

IN PROGRESS

High

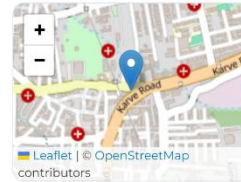
ADDRESS / LOCATION

Selected: 18.50190, 73.81474

DESCRIPTION

The public waste bin near the residential society gate has not been cleared for three days. Garbage is spilling onto the street and causing a foul odor. Address :- Kothrud

LOCATION MAP



YOUR EVIDENCE



Official Resolution

ADMIN COMMENT

The solid waste management team has been notified and a cleanup crew has been dispatched to this location.

PROOF OF FIX



Civic Pulse

Official Complaint Acknowledgement

Date: 3/24/2026

ID: #CP-2

ISSUE TITLE

Overflowing Public Garbage Bin

CATEGORY

Garbage

PRIORITY LEVEL

Medium

STATUS

Submitted

LOCATION / ADDRESS

Selected: 18.50190, 73.81474

COMPLAINT DESCRIPTION

The public waste bin near the residential society gate has not been cleared for three days. Garbage is spilling onto the street and causing a foul odor. Address :- Kothrud

OFFICIALLY RECEIVED

This document serves as proof that your complaint has been registered with the Civic Pulse Authority. Please retain this receipt for your records.

6. Testing

6.1 Test Plan

The testing strategy for Civic Pulse was designed to ensure system correctness, geographic accuracy, and security. Testing was conducted iteratively throughout the development lifecycle, focusing heavily on role-based access control (RBAC), database constraints, and frontend user interface behavior.

Testing Scope (Pages Evaluated):

- **Authentication Gateway:** Login and Registration pages (validation of credentials and routing).
- **Citizen Dashboard:** User profile, gamification leaderboard, and "My Complaints" tracking table.
- **Issue Reporting Page:** The complaint submission form and Leaflet.js interactive map integration.
- **Admin Analytics Dashboard:** System statistics and dynamic Chart.js rendering.
- **Admin Management Console:** The complaint feed, status update filters, and resolution image upload module.

6.2 Black Box Testing / Data Validation

Black Box testing was performed to validate system inputs and frontend GUI behaviors without examining the internal backend code.

- **User Authentication Validation:**
 - *Input:* Valid email and password -> *Output:* Successfully routed to the appropriate User or Admin dashboard.
 - *Input:* Incorrect password -> *Output:* System denies entry and displays an "Invalid Credentials" error message.
- **File Upload Validation:**
 - *Input:* Uploading a .jpg or .png image as evidence -> *Output:* File is accepted, securely renamed, and mapped to the database.
 - *Input:* Uploading a .pdf or .txt file -> *Output:* System rejects the file format and alerts the user to upload a valid image.
- **Form & Map Validation:**
 - *Input:* Submitting a complaint without clicking the map to drop a location pin -> *Output:* Form submission is halted; the user is prompted to provide geographic coordinates.

6.3 White Box Testing / Functional Validation

White Box testing was conducted to verify the internal logic, database queries, and backend Python (Flask) routing mechanisms.

- **Security & Session Logic Testing:**
 - Verified that standard citizens attempting to manually access the /admin_dashboard URL are automatically blocked and redirected to the login page (Session Verification).
 - Verified that all user passwords are cryptographically hashed using Werkzeug.security before being stored in the PostgreSQL database.
- **Gamification Logic Testing:**
 - Verified the arithmetic logic: When an administrator marks a complaint as "Resolved," the system correctly fetches the user's ID and adds +10 points to their database record.
 - Verified that rejecting a false or spam report correctly deducts points from the user's account.
- **Database Constraint Testing:**
 - Tested the Foreign Key ON DELETE CASCADE rule: Verified that deleting a user account automatically deletes all associated complaints to prevent orphaned data from crashing the application.

6.4 Test Results

All defined test cases were executed successfully. The Civic Pulse system met all expected behavioral and functional outcomes:

- The login routing strictly and securely separates Citizens from Administrators.
- The Leaflet.js map accurately captures float coordinates (Latitude/Longitude) and passes them to the backend without data loss.
- The system effectively blocks unauthorized file types and protects restricted URL routes.
- The platform demonstrated stable performance without memory leaks when loading interactive Chart.js widgets and querying the PostgreSQL database.

7. Conclusion and Recommendations

7.1 Conclusion

The Civic Pulse project successfully meets its objectives by delivering an efficient, secure, and user-friendly e-governance platform.

- Automates traditional, manual municipal grievance reporting into a centralized digital system.
- Integrates interactive GIS mapping (Leaflet.js) for precise location tracking of infrastructure issues.
- Boosts public participation through a gamified reward and ranking system.
- Empowers administrators with real-time visual analytics (Chart.js) and streamlined issue management.
- Fosters a transparent, secure, and highly responsive smart-city ecosystem.

7.2 Recommendations

- **Periodic Backups:** Schedule regular automated backups of the PostgreSQL database to secure citizen data and uploaded evidence.
- **Staff Training:** Conduct brief training sessions for municipal workers to efficiently utilize the administrative dashboard.
- **Security Updates:** Regularly update the Python framework and system dependencies to maintain platform security.

8. Future Scope

While the current version is fully functional, the system architecture allows for significant future enhancements:

- **Native Mobile Apps:** Develop Android and iOS applications for direct camera access and highly accurate mobile GPS tracking.
- **AI Image Verification:** Implement Machine Learning models to automatically scan and verify uploaded photos, reducing spam reports.
- **Predictive Analytics:** Upgrade the dashboard to forecast potential infrastructure failures based on historical data.
- **Cloud Deployment:** Migrate the local server to a cloud hosting platform (e.g., AWS or Google Cloud) to handle city-wide traffic scalability.
- **Multilingual Support:** Add regional language options to increase platform accessibility for all demographics.